

Tecnatura Superior en Desarrollo de Software

Módulo 04 — Capa Application

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

Antes de comenzar — lectura complementaria recomendada

Este módulo introduce conceptos que se apoyan en características del lenguaje C# que quizás no hayas utilizado aún con profundidad. Antes de continuar, o mientras avanzas, consultá los siguientes anexos:

Anexo	Temas	Cuándo leerlo
Anexo 01A — Interfaces y clases abstractas	Qué es una interfaz, sintaxis, diferencia con clase abstracta, contratos, implementación explícita, interfaces múltiples.	Antes de la Sección 3 de este documento
Anexo 01B — Herencia, constructores y métodos	Herencia de clases, constructores con parámetros, encadenamiento de constructores, métodos de instancia vs estáticos, modificadores de acceso.	Antes de la Sección 6 de este documento

¿Qué es la capa Application?

En Clean Architecture, la capa **Application** es la responsable de orquestar los casos de uso del sistema. No contiene reglas de negocio — esas viven en Domain — pero sí contiene la lógica que coordina cómo se ejecuta cada operación: qué datos se reciben, qué objetos de dominio se crean, qué repositorio se consulta y qué respuesta se devuelve.

Pensá en Application como el *director de orquesta*: no toca ningún instrumento, pero decide quién toca, cuándo y en qué orden. Los instrumentos son el Dominio (que valida y modela) y la Infraestructura (que persiste).

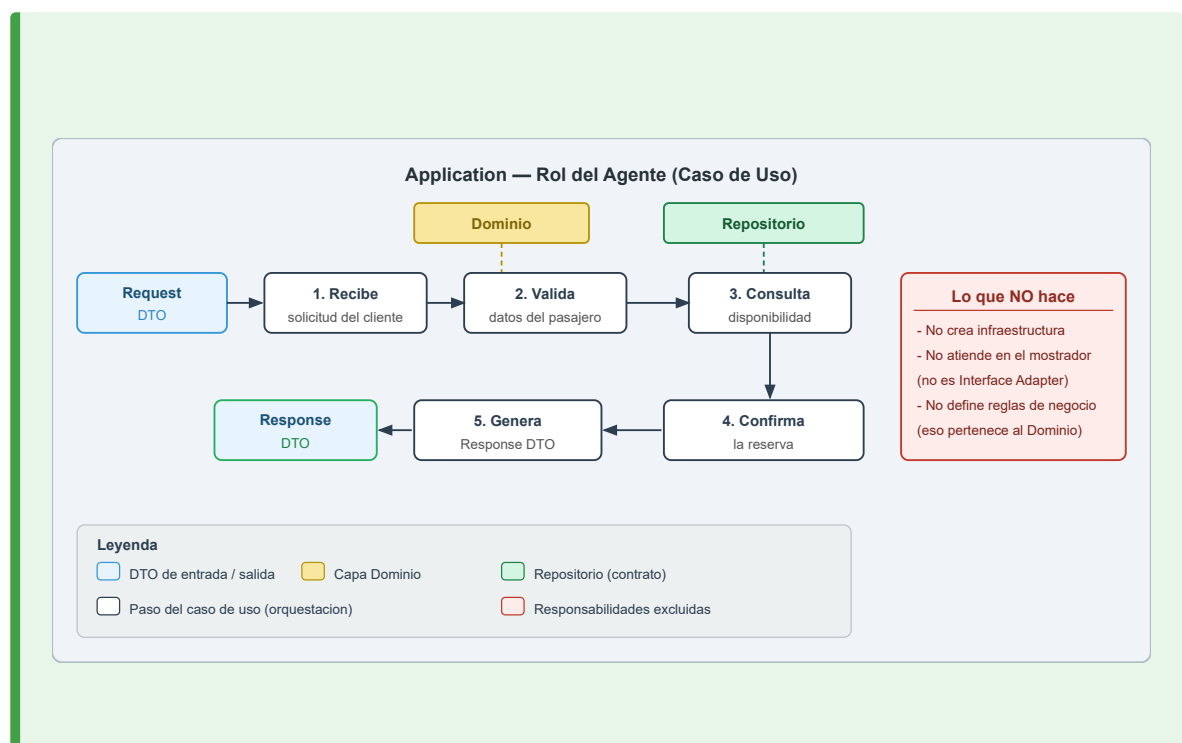
Los componentes centrales que viven en esta capa son: los **puertos de entrada** (`InputPort`), los **puertos de salida** (`OutputPort`), el **contrato de repositorio** y los **objetos de transferencia de datos** (`DTO`). Todos ellos se articulan alrededor de un único caso de uso implementado en una clase sellada.

Regla fundamental de Application:

La capa Application conoce el Dominio pero no conoce la Infraestructura ni la UI. Depende de abstracciones (interfaces), nunca de implementaciones concretas.

Analogía del mundo real

Imaginá una agencia de viajes. El cliente (Interface Adapter) llama al agente y le pide un pasaje a Roma con determinadas condiciones. El agente (Application) no construye el avión ni gestiona el aeropuerto (eso es Infraestructure), pero sí:



Los **puertos** son los contratos del formulario: el `InputPort` define qué información acepta el agente, el `OutputPort` define cómo comunica el resultado. Si cambia la

aerolínea (Infrastructure), el agente no cambia su formulario. Si cambia el mostrador (Interface Adapter), el agente tampoco cambia.

Características esenciales de la capa Application

Lectura recomendada: Para comprender las interfaces que se presentan en esta sección, consultá el **Anexo 01A — Interfaces y clases abstractas**.

Característica	Descripción
Sin dependencias hacia afuera	Application no referencia proyectos de Infrastructure ni de InterfaceAdapter. La dependencia siempre apunta hacia adentro: Application conoce Domain, nunca al revés.
Depende de abstracciones	Application define interfaces (<code>IOrderRepository</code> , <code>IPlaceOrderOutputPort</code>) y depende de ellas. Las implementaciones concretas viven en otras capas.
Orquesta, no valida	Application no re-valida los datos que ya validó el Dominio. Su responsabilidad es coordinar el flujo, no duplicar reglas.
DTOs como frontera	Los objetos que entran y salen del caso de uso son DTOs — tipos simples sin lógica de negocio. Esto desacopla la capa de quien la llama.
Un caso de uso por clase	Cada operación del sistema tiene su propia clase de caso de uso. <code>PlaceOrderUseCase</code> solo registra pedidos. Principio de Responsabilidad Única aplicado al nivel de caso de uso.

¿Por qué se usa este patrón?

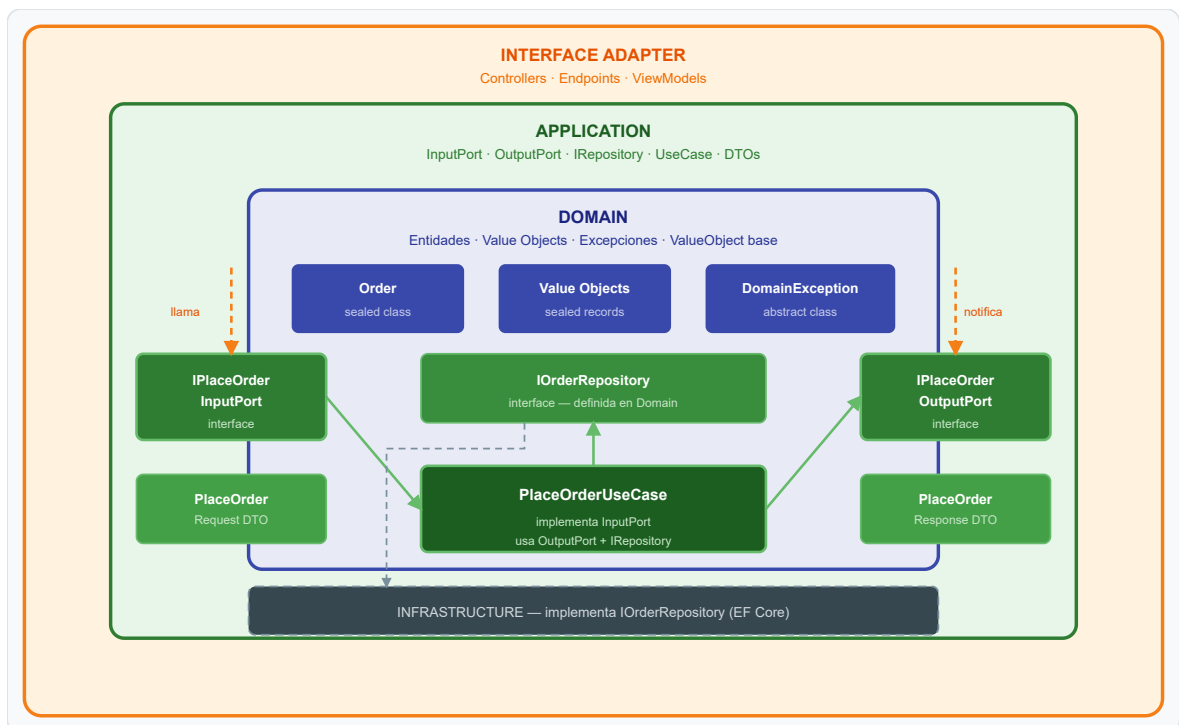
El patrón **Ports & Adapters** (también llamado *Arquitectura Hexagonal*) fue propuesto por Alistair Cockburn para resolver un problema recurrente: el código de negocio terminaba acoplado a la tecnología que lo rodeaba (base de datos, framework web, UI). Cuando la tecnología cambiaba, había que reescribir la lógica de negocio.

La solución es definir **puertos** — interfaces que describen cómo el sistema se comunica con el mundo exterior — y **adaptadores** — implementaciones concretas que conectan esos puertos con tecnologías específicas. El núcleo del sistema (Domain + Application) nunca sabe qué adaptador está activo.

Ventaja	Explicación
Testabilidad	El caso de uso puede probarse con un repositorio falso (mock) sin necesidad de base de datos real. Los puertos permiten sustituir cualquier dependencia.
Intercambiabilidad	Cambiar de SQL Server a PostgreSQL solo requiere una nueva implementación de <code>IOrderRepository</code> . El caso de uso no se modifica.
Independencia de frameworks	Application no depende de ASP.NET, Entity Framework ni ningún framework externo. Depende solo del Dominio y de sus propias interfaces.
Claridad del flujo	El InputPort define exactamente qué acepta el caso de uso. El OutputPort define exactamente qué devuelve. El contrato es explícito y verificable por el compilador.
Principio DIP	Application depende de abstracciones (interfaces), no de implementaciones concretas. Esto es el Principio de Inversión de Dependencias aplicado a nivel de arquitectura.

¿Cómo encaja en la arquitectura?

En Clean Architecture, las dependencias siempre apuntan hacia el centro. Application es la segunda capa desde el centro: conoce Domain pero no conoce Infraestructura ni InterfaceAdapter. Los puertos son la frontera entre capas. El diagrama siguiente muestra cómo interactúan los componentes en un caso de uso típico de registro de un pedido.



Componente	Capa	Tipo C#	Responsabilidad
IPlaceOrderInputPort	Application	interface	Contrato de entrada del caso de uso — lo llama InterfaceAdapter
IPlaceOrderOutputPort	Application	interface	Contrato de salida — lo implementa InterfaceAdapter
IOrderRepository	Domain	interface	Contrato de persistencia — lo implementa Infrastructure
PlaceOrderRequest	Application	class (DTO)	Datos crudos que entran al caso de uso
PlaceOrderResponse	Application	class (DTO)	Resultado que sale del caso de uso
PlaceOrderUseCase	Application	sealed class	Implementa InputPort — orquesta Domain + Repository + OutputPort

Marco teórico complementario

Lectura recomendada: Para comprender la herencia, constructores y métodos que se usan en esta sección, consultá el **Anexo 01B — Herencia, constructores y métodos**.

1 — DTO: Data Transfer Object

Un **DTO** es un objeto cuya única responsabilidad es transportar datos entre capas. No tiene lógica de negocio, no valida, no transforma. Es una estructura plana que representa la forma de los datos en la frontera entre dos capas.

Para registrar un pedido se necesitan dos DTOs: uno para los datos que ingresan al sistema (`PlaceOrderRequest`) y otro para el resultado que el caso de uso devuelve (`PlaceOrderResponse`). Los datos de entrada son primitivos (`string`, `int`, `DateOnly`) porque la validación ocurre dentro del Dominio, no en el DTO.

```
1 namespace CommerX.Application.Orders.DTOs;
2
3 // DTO de entrada – transporta datos crudos desde InterfaceAdapter hacia el caso de uso
4 // class (no record): sin lógica de negocio, sin igualdad estructural necesaria
5 public class PlaceOrderRequest
6 {
7     // propiedades requeridas – init: solo asignables en la construcción
8     public required string ProductCode { get; init; }
9     public required int Quantity { get; init; }
10    public required string CustomerCode { get; init; }
11    public required DateOnly DeliveryDate { get; init; }
12 }
```

El DTO de salida transporta únicamente los datos que el caso de uso confirma una vez que la operación fue ejecutada con éxito. No incluye lógica ni referencias a entidades de dominio — solo primitivos.

```
1 namespace CommerX.Application.Orders.DTOs;
2
3 // DTO de salida – transporta el resultado del caso de uso hacia InterfaceAdapter
4 public class PlaceOrderResponse
5 {
6     // identificador generado por el dominio – confirma que el pedido fue registrado
7     public required Guid Id { get; init; }
```

```
8     public required string ProductCode { get; init; }
9     public required int Quantity { get; init; }
10    // fecha y hora UTC en que se registró el pedido
11    public required DateTime CreatedAt { get; init; }
12 }
```

2 — InputPort: el contrato de entrada

El **InputPort** es una interfaz que define el único método que el caso de uso expone al mundo exterior. InterfaceAdapter llama a este método sin saber nada de la implementación. Esto permite que el caso de uso sea reemplazado o mockeado sin cambiar una sola línea del controlador.

El método se declara asincrónico (`Task`) porque la operación implica acceso a la base de datos a través del repositorio. InterfaceAdapter solo conoce esta interfaz — nunca la clase concreta.

```
1     using CommerX.Application.Orders.DTOs;
2
3     namespace CommerX.Application.Orders.Ports;
4
5     // contrato de entrada del caso de uso – registrar un pedido
6     // quien llame a Execute() no sabe cómo está implementado – solo qué recibe
7     public interface IPlaceOrderInputPort
8     {
9         // Task: operación asincrónica – el repositorio persiste de forma async
10        Task Execute(PlaceOrderRequest request);
11    }
```

3 — OutputPort: el contrato de salida

El **OutputPort** es una interfaz que define cómo el caso de uso comunica su resultado. En lugar de retornar un valor directamente, el caso de uso llama a un método del OutputPort. Esto permite que el mismo caso de uso pueda presentar el resultado de formas completamente distintas (JSON, HTML, consola) sin modificar una sola línea de Application.

El OutputPort declara un método por cada resultado posible: éxito y error de validación. InterfaceAdapter implementa esta interfaz y decide cómo transformar cada resultado en una respuesta HTTP, un mensaje en pantalla o cualquier otra representación.

```
1     using CommerX.Application.Orders.DTOs;
```

```
2
3 namespace CommerX.Application.Orders.Ports;
4
5 // contrato de salida del caso de uso – registrar un pedido
6 // InterfaceAdapter implementa esta interfaz para recibir el resultado
7 public interface IPlaceOrderOutputPort
8 {
9     // éxito: el pedido fue registrado – se recibe la respuesta con los datos
10    void Success(PlaceOrderResponse response);
11
12    // fallo de validación de dominio – se recibe el mensaje de la excepción
13    void ValidationErrorMessage(string message);
14 }
```

4 — IOrderRepository: el contrato de persistencia

El repositorio es el contrato que Application usa para leer y escribir entidades en la base de datos. Vive en **Domain** — no en Application — porque es el dominio quien define qué operaciones de persistencia necesita, no la infraestructura. Infraestructure lo implementa con EF Core, pero Application y Domain solo ven la interfaz.

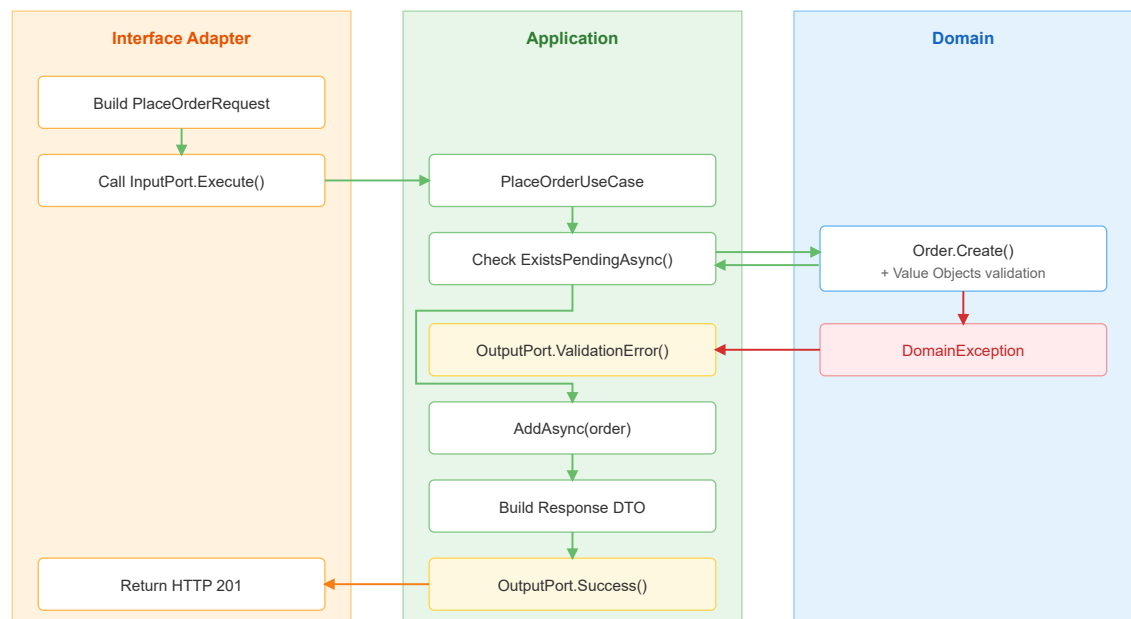
Cada método del repositorio es asíncrono porque implica una operación de I/O. Los parámetros son primitivos o entidades de dominio — nunca DTOs ni tipos de EF Core. El repositorio trabaja con el lenguaje del dominio.

```
1 using CommerX.Domain.Orders.Entities;
2
3 namespace CommerX.Domain.Orders.Repositories;
4
5 // contrato de persistencia – Infraestructure lo implementa con EF Core
6 // Application solo conoce esta interfaz – nunca la implementación concreta
7 public interface IOrderRepository
8 {
9     // agrega un pedido nuevo – async porque implica I/O de base de datos
10    Task AddAsync(Order order);
11
12    // verifica si ya existe un pedido pendiente para el mismo producto y cliente
13    Task<bool> ExistsPendingAsync(string productCode, string customerCode);
14 }
```

5 — El flujo completo del caso de uso

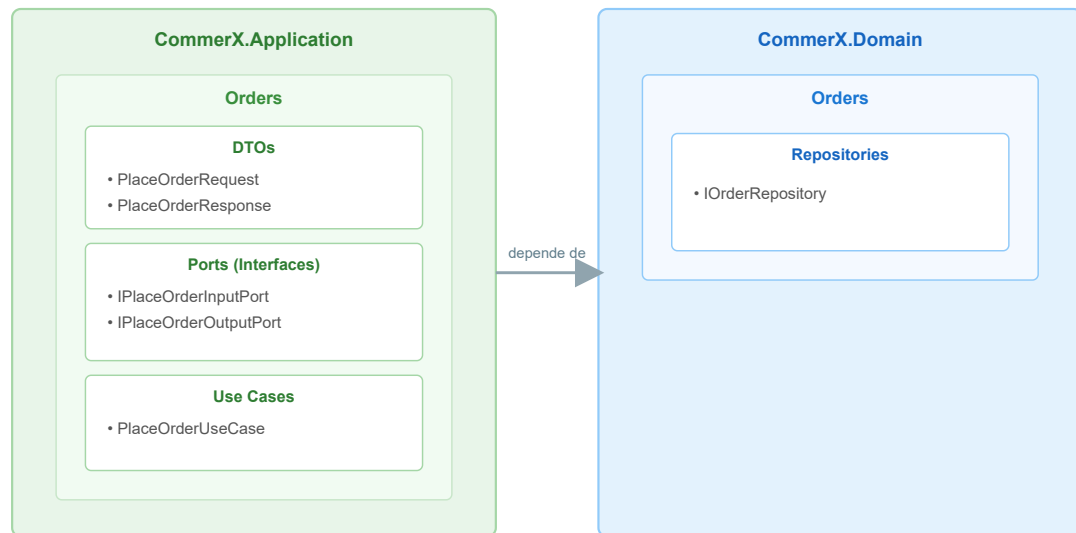
Con todos los contratos definidos, el flujo de ejecución queda completamente trazado. El caso de uso orquesta todos los componentes sin depender de ninguna

tecnología concreta. El siguiente diagrama muestra la secuencia de llamadas para registrar un pedido:



6 — Estructura de carpetas de Application

La organización de carpetas refleja la separación de responsabilidades: DTOs, Ports y UseCases son carpetas distintas dentro del módulo funcional (`Orders`). El repositorio vive en Domain porque es el dominio quien define qué necesita persistir.



Errores comunes

Error	Consecuencia
Validar en el caso de uso lo que ya validó el dominio	Duplica lógica de negocio. Si cambia una regla de validación, hay que cambiarla en dos lugares. Viola SRP.
Retornar la entidad de dominio desde el caso de uso	Expone el modelo de dominio a capas externas. InterfaceAdapter no debería conocer <code>Order</code> — solo el DTO.
Poner lógica de negocio en el DTO	Un DTO con métodos de validación viola SRP. La validación vive en el Dominio — los DTOs solo transportan datos.
Implementar el repositorio en Application	Application define el contrato, Infrastructure lo implementa. Si Application implementa el repositorio, depende de EF Core y viola el principio de independencia de frameworks.
Usar <code>record</code> para los DTOs	Según las reglas del proyecto, los DTOs son <code>class</code> — no <code>record</code> . Los records activan igualdad estructural que los DTOs no necesitan.
Omitir el OutputPort y retornar directamente	Si el caso de uso retorna el response directamente, el presenter (InterfaceAdapter) queda acoplado al tipo de retorno. Con

Error	Consecuencia
	OutputPort, el mismo caso de uso puede notificar a distintos presenters sin modificarse.

Qué NO hacer — código incorrecto

Antipatrón 1 — Validación duplicada en el caso de uso

El caso de uso re-valida datos que el método `Order.Create()` ya va a validar internamente a través de sus Value Objects. Duplicar esa lógica viola SRP: si cambia la regla de cantidad mínima, hay que actualizar dos lugares en lugar de uno.

```
1 // MAL: el caso de uso re-valida lo que ya hace el dominio
2 public async Task Execute(PlaceOrderRequest request)
3 {
4     // MAL: validación duplicada - Quantity.Create() ya verifica esto en el dominio
5     if (request.Quantity <= 0)
6         throw new Exception("La cantidad debe ser mayor a cero");
7
8     // el dominio ya valida internamente - la verificación de arriba es redundante
9     var order = Order.Create(request.ProductCode, request.Quantity, ...);
10 }
```

Antipatrón 2 — Retornar la entidad de dominio

El InputPort declara en su firma que retorna la entidad `Order` en lugar de un DTO. Esto obliga a InterfaceAdapter a conocer el modelo de dominio, generando acoplamiento directo entre capas que deberían estar desconectadas.

```
1 // MAL: la firma del puerto retorna la entidad - expone el dominio al exterior
2 public interface IPlaceOrderInputPort
3 {
4     // MAL: retorna Order - InterfaceAdapter queda acoplado al modelo de dominio
5     Task<Order> Execute(PlaceOrderRequest request);
6 }
```

Antipatrón 3 — DTO con lógica de negocio

El DTO incluye un método que valida o transforma sus propios datos. Viola SRP: un DTO es un contenedor de datos, no un objeto con comportamiento. Cualquier

lógica de validación pertenece al Value Object correspondiente en el Dominio.

```
1 // MAL: el DTO incluye lógica – la validación pertenece al dominio
2 public class PlaceOrderRequest
3 {
4     public int Quantity { get; set; }
5
6     // MAL: lógica de negocio dentro de un DTO – viola SRP
7     public bool IsQuantityValid() => Quantity > 0;
8 }
```